

How Computers Work · Section 9

The Boot Process

By TOUCH DEN · November 11, 2026

Contents

What You Will Learn 1

Before You Start 1

1. From Power Button to Desktop 1

2. Firmware: BIOS and UEFI 2

3. The Bootloader 2

4. Loading the Kernel 3

5. Init and the First Processes 3

6. Where Malware Hides at Boot 4

7. Secure Boot 5

Common Mistakes 5

Summary 6

Check Your Understanding 6

Practice 7

What You Will Learn

- What happens between pressing the power button and reaching the desktop.
- Firmware: BIOS and UEFI.
- The bootloader.
- How the kernel is loaded.
- Init, and the first processes.
- Where malware hides at boot, and why that is so dangerous.
- Secure Boot, and how it defends the start of the chain.

Before You Start

Read lessons 4, 5, and 6. You know about the CPU, the OS and its kernel, and processes. This lesson explains how a cold, powered-off machine becomes a running operating system, the **boot process**, a chain of steps where each stage starts the next. It matters because attackers who get in early, before the OS, gain enormous and hidden power. No code; one small command shows how long a machine has been running since boot.

---

1. From Power Button to Desktop

When you press the power button, the computer is a lump of hardware with nothing running, RAM is empty (it is volatile, lesson 4), and the operating system is just a file on the disk, not yet loaded. Somehow, in a few seconds, you reach a running desktop. That journey is the **boot process** (from "bootstrapping", pulling yourself up by your own bootstraps, because the computer must start itself from almost nothing).

The whole thing is a **chain**: each stage is small and simple, and its main job is to find, check, and start the *next*, bigger stage. Step by step:

1. **Firmware** wakes up the hardware and finds a drive to boot from.
2. The **bootloader** on that drive is started, and it finds and loads the kernel.
3. The **kernel** (the OS core, lesson 5) takes over and sets up the system.
4. The **init** process starts the first user-space programs, leading to the login screen and desktop.

Each link hands control to the next. And that is exactly why boot matters for security: **whoever controls an early link controls everything that comes after it.** A chain is only as trustworthy as its first link. The rest of this lesson walks the chain, then shows what happens when an attacker inserts themselves into it.

2. Firmware: BIOS and UEFI

The very first code to run is not on the hard drive at all, it is **firmware**: a small, permanent program stored on a chip on the computer's motherboard. Because it is on a chip, it is there the instant power arrives, before any disk is read. (Firmware in general is software baked into hardware; it is a whole security field of its own, the IoT and Hardware course.)

The firmware's job at power-on:

- **Wake and test the hardware** (the CPU, RAM, and essential devices). You may have seen this as a logo or a memory check.
- **Find a bootable drive** (a disk, SSD, or USB) in a configured order.
- **Load and run the first piece of software from that drive**, the bootloader (topic 3).

This firmware has two generations you should know by name. **BIOS** (Basic Input/Output System) is the old standard, decades old and now largely replaced. **UEFI** (Unified Extensible Firmware Interface) is the modern replacement on nearly all current computers: more capable, and importantly, able to perform security checks on what it boots (Secure Boot, topic 7). When people say "enter the BIOS" to change boot settings, they usually mean the UEFI firmware settings today.

Firmware is powerful and runs before the OS, which makes it a prized target. Malware that infects firmware (sometimes called a **bootkit** at the firmware level) runs before the operating system and any antivirus, and can survive reinstalling the OS or even replacing the hard drive, because it does not live on the drive. Such attacks are advanced and rare, but they show the principle: the earlier the attacker is in the chain, the harder they are to see and remove.

3. The Bootloader

The firmware does not know how to start a whole operating system; that is too complex for it. So it hands off to a specialist: the **bootloader**, a small program whose one job is to find the operating system's kernel on the drive, load it into memory, and start it.

Because a computer can have more than one operating system installed, the bootloader is also what sometimes shows a menu ("start Windows, or Linux?") and lets you choose. Common bootloaders include GRUB on Linux and the Windows Boot Manager. Whatever the name, the

role is the same: the bridge between the firmware (which knows the hardware) and the kernel (which knows the OS).

The bootloader is another sensitive link, for the same reason as firmware: it runs **before** the kernel and decides *which* kernel to load. An attacker who tampers with the bootloader can load a modified or malicious kernel, or quietly insert code, before any OS protection exists. This class of attack is called a **bootkit**. It is powerful precisely because it sits below the operating system, so the OS's own defenses, running later, may never see it. Guarding the bootloader's integrity is a key reason Secure Boot exists.

4. Loading the Kernel

The bootloader loads the **kernel** into memory and transfers control to it. From lesson 5, the kernel is the privileged core of the operating system, and now it takes charge of the machine for the rest of the session. Its startup work includes:

- **Initializing the hardware** properly, loading the **drivers** (lesson 5) it needs to talk to the disk, network, graphics, and other devices.
- **Setting up memory management**, including the virtual-memory system (lesson 6) that will isolate every process.
- **Preparing to run programs**, then starting the very first user-space process (topic 5).

This is the moment the real operating system comes alive. Everything you learned about the kernel, system calls, user and kernel space, process isolation, begins here, as the kernel builds the managed, protected environment that all your programs will run inside.

For security, note the handover of trust: the bootloader vouched for the kernel and started it with full privilege. If every earlier link was trustworthy, the kernel that now runs is the genuine one. If an earlier link was compromised, the "kernel" now in charge might be an attacker's version, with total control and nothing above it to object. This is why the integrity of the whole chain, firmware → bootloader → kernel, is treated as one connected security problem.

5. Init and the First Processes

A running kernel manages programs, but at this instant there are none, no login screen, no desktop, nothing in user space. So the kernel starts one special first program, traditionally called **init** (on modern Linux it is usually **systemd**; other systems have their equivalents). This is the ancestor of every other process.

Recall from lesson 6 that every process has a parent (its PPID), forming a family tree. **Init is the root of that tree**: it is the first user-space process, and it starts the next ones, the background services, the login screen, and eventually the programs you open, each of which starts more. Trace any process's parents back far enough and you arrive at init. This is also why init has a famous process ID: **PID 1**.

From here, the system reaches the state you recognize: background services running, a login prompt, and after you log in, your desktop and apps. All of it descends from init, which descended from the kernel, which the bootloader loaded, which the firmware started. The chain is complete, and your computer has "booted".

You can see the result of a successful boot with a simple command, `uptime`, which reports how long the machine has been running since it booted:

```
$ uptime
22:32  up 6 days, 23:47, 1 user, load averages: 1.31 1.43 1.70
```

"up 6 days" means the whole chain, firmware to desktop, completed almost a week ago and the system has run since. Every program on it traces back through that boot.

6. Where Malware Hides at Boot

Now the security payoff. The boot chain is attractive to attackers for one reason above all: **persistence**. A piece of malware wants to survive restarts, if it only lived in RAM, a reboot would wipe it (lesson 4). So malware seeks a foothold in the boot process, so it is loaded automatically every time the machine starts, early, and often invisibly.

The earlier in the chain the malware sits, the more dangerous it is:

Where it hides	Name	Why it is dangerous
Firmware (the chip)	Firmware implant / bootkit	Runs before the OS; survives OS reinstall and even disk replacement
Bootloader	Bootkit	Runs before the kernel; can load a tampered kernel; OS defenses may never see it
Kernel (as a driver)	Rootkit	Runs with full privilege; can hide itself, files, and processes from the OS
Startup programs/services	Autostart persistence	Common and easier to find; runs after boot as a normal (if hidden) program

The deeper ones, bootkits and rootkits, are so serious because they load **before** the tools that would detect them, and can then **lie** to those tools, hiding their own files and processes. This is a central theme of Malware Analysis (course 29) and a reason forensic investigators sometimes examine a disk from a *separate*, trusted machine: if the suspect system's own OS is compromised, you cannot trust what it tells you about itself. The principle from topic 1 returns with full force: control an early link, and you control, and can hide from, everything later.

7. Secure Boot

If the danger is a tampered early link, the defense is to **check each link before trusting it.** That is exactly what **Secure Boot** does. Built into UEFI firmware (topic 2), Secure Boot uses digital signatures (a cryptography idea you will study in course 09) to verify that each stage of the boot is genuine and unmodified before running it.

The idea is a **chain of trust**: the firmware only runs a bootloader that carries a valid, trusted signature; that bootloader only loads a kernel that is properly signed; and so on. If any link has been tampered with, its signature no longer matches, and Secure Boot refuses to continue, stopping a bootkit before it runs. Each link verifies the next, so trust extends from the firmware all the way up to the OS.

Related technologies (such as a **TPM**, a Trusted Platform Module chip, and "measured boot") go further, recording what was loaded so the system can prove it booted in a known-good state. You do not need the details now. The concept to carry forward is the important one: because the boot chain is only as trustworthy as its first link, modern computers try to **verify every link before trusting it**, turning a vulnerable chain into a defended one. This is defense in depth (Security Fundamentals) applied to the very start of a computer's life.

You now understand a computer's full life: from power-on through firmware, bootloader, kernel, and init, to a running system, and why each early step is a security battleground. Next, we look at a technology that lets one physical computer safely run many separate systems at once, the foundation of your future practice lab: virtual machines and containers.

Common Mistakes

1. Thinking the OS is the first thing to run. Firmware runs first, then the bootloader, then the kernel. The OS is loaded late in the chain.

2. Underestimating early-stage malware. Bootkits and firmware implants run before the OS and its antivirus, can survive reinstalls, and can hide from the running system. Earlier = more dangerous.

3. Trusting a compromised machine to report on itself. A rootkit can lie to the OS's own tools. Serious forensics examines the disk from a separate, trusted system.

4. Confusing firmware with the OS. Firmware is on a chip on the motherboard and runs at power-on; the OS is a file on the disk, loaded later.

5. Ignoring Secure Boot as "just an annoyance". It verifies each boot stage's signature to stop tampering early, a real defense, not a formality.

Summary

Term	Remember
Boot process	The chain from power-on to a running OS; each stage starts the next
Firmware (BIOS/UEFI)	On a motherboard chip; runs first; wakes hardware and starts the bootloader
Bootloader	Finds, loads, and starts the kernel; runs before the OS
Kernel load	The kernel takes over, loads drivers, sets up memory, starts init
Init (PID 1)	The first user-space process; ancestor of all others
Boot malware	Bootkits/rootkits run early, survive reboots, and hide; persistence is the goal
Secure Boot	UEFI verifies each stage's signature (chain of trust) before running it

Check Your Understanding

1. What is the boot process, and why is it described as a chain?
2. What is firmware, where is it stored, and what are BIOS and UEFI?
3. What is the bootloader's one main job?
4. What does the kernel set up as it starts, and why does that connect to earlier lessons?
5. What is init, what is its PID, and why is it special?
6. Why is malware that hides in firmware or the bootloader so much more dangerous than an ordinary startup program?
7. What does Secure Boot do, and what is a "chain of trust"?

Answers

1. The sequence of steps from pressing power to a running OS. It's a chain because each stage's job is to find, check, and start the next, bigger stage, so control passes link to link.
2. Firmware is a small permanent program on a chip on the motherboard, running the instant power arrives. BIOS is the old standard; UEFI is the modern replacement that can also verify

what it boots (Secure Boot).

3. To find the operating system's kernel on the drive, load it into memory, and start it.
4. It initializes hardware and loads drivers, sets up memory management including virtual memory, and starts the first user-space process. This is where the kernel, system calls, and process isolation from earlier lessons begin.
5. Init is the first user-space process the kernel starts; its PID is 1; it's special because it's the ancestor of every other process, the root of the process tree.
6. Because it runs before the OS and its antivirus even load, can survive OS reinstalls (and firmware implants survive disk replacement), and can hide itself from the running system's tools. Earlier in the chain means more control and better concealment.
7. Secure Boot (in UEFI) uses digital signatures to verify each boot stage is genuine and unmodified before running it. A chain of trust means each link only runs the next if its signature is valid, so trust extends from firmware up to the OS.

Practice

1. Put these boot stages in order, and say in one phrase what each does: kernel, firmware, init, bootloader.
2. Explain to a friend why malware that infects firmware is harder to remove than a virus that is just an app, using what you know about where each lives.
3. If you have a terminal, run `uptime` (Linux/macOS). How long has your machine been running since it last completed the boot chain?

A Worked Example

1. **Firmware** (wakes the hardware, finds a boot drive) → **bootloader** (finds and loads the kernel) → **kernel** (takes over, sets up the system) → **init** (starts the first user programs, leading to login and desktop).
2. "A normal virus is a file on the disk, so wiping or reinstalling the disk removes it. Firmware malware lives on a chip on the motherboard, not the disk, and runs before the operating system even loads, so reinstalling the OS, or even swapping the hard drive, doesn't touch it, and the OS's antivirus never gets a chance to see it."
3. For example, `uptime` showing "up 6 days" means the machine completed its boot almost a week ago and has run continuously since; every process on it traces back to that boot.

Course mark icon from Material Symbols by Google, Apache License 2.0.